

Lucrarea nr. 1**Reprezentare și manipularea informației grafice în format binar,
nivele de gri, color****Formatul grafic BMP**

În acest laborator vom examina formatul imagine BMP.

După ce fotografia a fost captată, este stocată într-un format digital universal care permite să fie transferată la calculator, imprimantă sau TV. Există o mare varietate de formate de memorare a imaginii. Dintre cele mai des folosite se pot aminti: BMP, TIFF, EPS, JPG, PNG, GIF, etc.

Structura BMP

Un fișier BMP conține următoarele structuri de date:

BITMAPFILEHEADER bmfh; (conține informații despre fișier, nu despre imagine în sine)

BITMAPINFOHEADER bmih; (conține informații despre bitmap cum ar fi: dimensiune, număr de culori, etc)

RGBQUAD aColors[]; (conține tabela de culori)

Componentele structurii BMP

În cadrul enumerării următoare am respectat denumirile câmpurilor așa cum apar ele în documentația originală.

Structura **BITMAPFILEHEADER** este prezentată în tabelul 1.

Tabelul 1 – Structura BITMAPFILEHEADER				
Start	Dimensiune (octeți)	Nume	Valoare standard	
1	2	bfType	19778	
3	4	bfSize	????	
7	2	bfReserved1	0	
9	2	bfReserved2	0	
11	4	bfOffBits	????	

Structura **BITMAPINFOHEADER** este prezentată în tabelul 2.

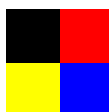
Tabelul 2 – Structura BITMAPINFOHEADER				
Start	Dimensiune (octeți)	Nume	Valoare standard	
15	4	biSize	40	
19	4	biWidth	100	
23	4	biHeight	100	
27	2	biPlanes	1	
29	2	biBitCount	????	
31	4	biCompression	0	
35	4	biSizeImage	????	
39	4	biXPelsPerMeter	0	
43	4	biYPelsPerMeter	0	
47	4	biClrUsed	0	
51	4	biClrImportant	0	

În cazul în care imaginile utilizează 4 sau 8 biți pe pixel trebuie să avem o paletă de culori. Numărul valid de culori cuprinse în această paletă variază după cum urmează: 16 și respectiv 256 dar până la urmă ele pot fi mai mici dacă imaginea nu face uz de toate culorile posibile. În cazul în care se folosesc 24 de biți pe pixel avem de-a face cu o imagine full-color care nu mai necesită o paletă de culori.

Structura **RGBQUAD** este prezentată în tabelul 3.

Tabelul 3 – Structura RGBQUAD				
Start	Dimensiune (octeți)	Nume	Valoare standard	
1	1	rgbBlue	??	
2	1	rgbGreen	??	
3	1	rgbRed	??	
4	1	rgbReserved	??	

Exemplu de fișier de imagine cu 4 pixeli:



Manipularea structurilor unui fișier BMP în limbajul C#

Aplicația propusă în acest laborator va citi și afișa în controale corespunzătoare conținutul structurilor unui fișier BMP (Fig. 3).

The screenshot shows a Windows application window titled "Form1" with three main sections for editing BMP file headers and a color palette.

- BITMAPFILEHEADER:**
 - bfType = 424d
 - bfSize = 46000000
 - bfReserved1 = 0000
 - bfReserved2 = 0000
 - bfOffBits = 36000000
- BITMAPINFOHEADER:**
 - biSize = 28000000
 - biWidth = 02000000
 - biHeight = 02000000
 - biPlanes = 0100
 - biBitCount = 1800
 - biCompresion = 00000000
 - biSizeImage = 10000000
 - biXPelsPerMeter = 00000000
 - biYPelsPerMeter = 00000000
 - biClrUsed = 00000000
 - biClrImportant = 00000000
- RGBQuad:** A text area containing the hex values for the color palette: 00 ff ff 00 00 00 00 00 00 00 00 ff 00 00.
- Color Palette:** A 2x2 grid of colored squares: Black, Red, Yellow, and Blue.
- Buttons:** "Citeste fisier bmp ca text" and "Salvare ca fisier bmp".

Fig.3 – Conținutul fișierului BMP

```
public struct BITMAPFILEHEADER
{
    public int bfType;
    public int bfSize;
    public short bfReserved1, bfReserved2;
    public int bfOffBits;
}

public struct BITMAPINFOHEADER
{
    public int biSize;
    public int biWidth, biHeight;
    public short biPlanes, biBitCount;
    public int biCompression, biSizeImage;
    public int biXPelsPerMeter, biYPelsPerMeter;
    public int biClrUsed, biClrImportant;
}

public struct RGBQUAD
{
    public int rgbBlue;
    public int rgbGreen;
    public int rgbRed;
    public int rgbReserved;
}

BITMAPFILEHEADER bfh = new BITMAPFILEHEADER();
BITMAPINFOHEADER bih = new BITMAPINFOHEADER();
RGBQUAD rgb = new RGBQUAD();
```

```

byte[] buffer = new byte[100];

private void CitesteFisierBMPTText_Click(object sender, EventArgs e)
{
    BinaryReader br = null;
    byte[] file = null;

    try
    {
        // Citim fisierul bmp
        br = new BinaryReader(File.OpenRead("C:\\untitled.bmp"));

        //citim in file datele
        file = br.ReadBytes((int)br.BaseStream.Length);

        //afisam datele din fisierul bmp in richTextBox
        for (int i = 0; i < file.Length; i++)
            richTextBox1.Text += Convert.ToString(file[i], 16).PadLeft(2, '0')
                                + " ";

        //le impartim pe structuri si le afisam in controale separate
        //afisam BITMAPFILEHEADER
        for (int i = 0; i < bfh.bfType; i++)
            txt_bftype.Text += Convert.ToString(file[i], 16).PadLeft(2, '0');
        for (int i = bfh.bfType; i < bfh.bfType+bfh.bfSize; i++)
            txt_bfsize.Text += Convert.ToString(file[i], 16).PadLeft(2, '0');
        for (int i = bfh.bfType + bfh.bfSize; i < bfh.bfType +
            bfh.bfSize+bfh.bfReserved1; i++)
            txt_bfreserved1.Text += Convert.ToString(file[i],
                16).PadLeft(2, '0');
        for (int i = bfh.bfType + bfh.bfSize + bfh.bfReserved1; i <
            bfh.bfType + bfh.bfSize + bfh.bfReserved1 + bfh.bfReserved2; i++)

        //se continua similar pentru restul campurilor din structura de date
        BITMAPFILEHEADER

        .....

        //SE VERIFICA CORECTITUDINEA FUNCTIONARII APLICATIEI PANA IN ACEST PUNCT

        //se continua cu distribuirea datelor pentru structura de date BITMAPINFOHEADER

        .....

        //SE VERIFICA CORECTITUDINEA FUNCTIONARII APLICATIEI PANA IN ACEST PUNCT

        //afisam continutul imaginii RGBQuad

        //citesc de la ce octet incolo incep datele din fisier
        string s = ""; //inversez ce am in textBox pt ca am ocmps urmat de
ocms
        for (int i = txt_bfoffbits.Text.Length-2; i >= 0; i--,i--)
        {
            s+=txt_bfoffbits.Text[i];
            s += txt_bfoffbits.Text[i+1];
        }
        converttobytes(s); //convert in bytes
        int x = buffer[0]; //pun intr-un intreg
        x = x << 8;
        x = buffer[1];
        x = x << 8;
        x = buffer[2];
        x = x << 8;
        x = buffer[3];
    }
}

```

```

        //citesc lungimea fisierului
        s = "";
        for (int i = txt_bfsize.Text.Length - 2; i >= 0; i--, i--)
        {
            s += txt_bfsize.Text[i];
            s += txt_bfsize.Text[i + 1];
        }
        converttobytes(s); //convert in bytes
        int x1 = buffer[0]; //pun intr-un intreg
        x1 = x1 << 8;
        x1 = buffer[1];
        x1 = x1 << 8;
        x1 = buffer[2];
        x1 = x1 << 8;
        x1 = buffer[3];

        //citesc numarul de linii din fisier
        s = "";
        for (int i = txt_biheight.Text.Length - 2; i >= 0; i--, i--)
        {
            s += txt_biheight.Text[i];
            s += txt_biheight.Text[i + 1];
        }
        converttobytes(s); //convert in bytes
        int x2 = buffer[0]; //pun intr-un intreg
        x2 = x2 << 8;
        x2 = buffer[1];
        x2 = x2 << 8;
        x2 = buffer[2];
        x2 = x2 << 8;
        x2 = buffer[3];

        //octeti de date
        int nrOctetiDate = x1 - x;

        //octetii ce trebuie afisati pe fiecare linie
        int octetiLinie = nrOctetiDate / x2;

        //afisez date (de pe fiecare linie) din fisierul bmp
        for (int i = x, j=0; i < x1; i++, j++)
        {
            if (j==octetiLinie)
            {
                j=0;
                txt_RGBQuad.Text += "\r\n";
            }
            txt_RGBQuad.Text += Convert.ToString(file[i], 16).PadLeft(2,
'0');
        }
    }
    catch (Exception exc)
    {
        // Afisam eroarea catre utilizator.
        MessageBox.Show("Fisierul nu a putut fi deschis sau citit" +
"Exception: " + exc.Message);
    }
    finally
    {
        // Inchidem
        if (br != null)
        {
            br.Close();
        }
    }
}

```

```
//salvam imaginea intr-un fisier bmp
private BinaryWriter dataOut;
private string numeFisier = "c:\\exemplu.BMP";
private void Save_Click(object sender, EventArgs e)
{
    byte[] fileOut = new byte[1000];

    int j = 0;

    converttobytes(txt_bftype.Text); //convert in bytes
    for (int i = 0; i < txt_bftype.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bfsize.Text); //convert in bytes
    for (int i = 0; i < txt_bfsize.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bfreserved1.Text); //convert in bytes
    for (int i = 0; i < txt_bfreserved1.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bfreserved2.Text); //convert in bytes
    for (int i = 0; i < txt_bfreserved2.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bfoffbits.Text); //convert in bytes
    for (int i = 0; i < txt_bfoffbits.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bisize.Text); //convert in bytes
    for (int i = 0; i < txt_bisize.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_biwidth.Text); //convert in bytes
    for (int i = 0; i < txt_biwidth.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_biheight.Text); //convert in bytes
    for (int i = 0; i < txt_biheight.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_biplanes.Text); //convert in bytes
    for (int i = 0; i < txt_biplanes.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bibitcount.Text); //convert in bytes
    for (int i = 0; i < txt_bibitcount.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bicompresion.Text); //convert in bytes
    for (int i = 0; i < txt_bicompresion.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bisizeimage.Text); //convert in bytes
    for (int i = 0; i < txt_bisizeimage.Text.Length / 2; i++, j++)
        fileOut[j] = buffer[i];

    converttobytes(txt_bixpelspermeter.Text); //convert in bytes
```

```

        for (int i = 0; i < txt_bixpelspermeter.Text.Length / 2; i++,
j++)
            fileOut[j] = buffer[i];

        converttobytes(txt_biypelspermeter.Text); //convert in bytes
        for (int i = 0; i < txt_biypelspermeter.Text.Length / 2; i++,
j++)
            fileOut[j] = buffer[i];

        converttobytes(txt_biclruled.Text); //convert in bytes
        for (int i = 0; i < txt_biclruled.Text.Length / 2; i++, j++)
            fileOut[j] = buffer[i];

        converttobytes(txt_biclrimportant.Text); //convert in bytes
        for (int i = 0; i < txt_biclrimportant.Text.Length / 2; i++,
j++)
            fileOut[j] = buffer[i];

        string[] s; string x=null;
        s= txt_RGBQuad.Text.Split(new String[]{"\r\n"}, Int32.MaxValue,
                                0);

        for (int i = 0; i < s.Length; i++)
        {
            x += s[i];
        }

        converttobytes(x); //convert in bytes
        for (int i = 0; i < x.Length / 2; i++, j++)
            fileOut[j] = buffer[i];

        dataOut = new BinaryWriter(new FileStream(numefisier,
FileMode.Create));
        dataOut.Close();

        dataOut = new BinaryWriter(new FileStream(numefisier,
FileMode.Append));
        dataOut.Write(fileOut);
        dataOut.Close();
    }

```

Desfășurarea lucrării:

1. Se va citi breviarul teoretic. Se atrage atenția asupra faptului că aceste informații vor fi folosite și la desfășurarea altor lucrări de laborator.
2. Se vor studia exemplele prezentate pe parcursul acestui laborator urmărind efectul rulării acestora și buna lor execuție.
3. Se va încerca găsirea altor posibilități de soluționare a soluțiilor propuse.
4. Să se creeze un fișier bmp ce va conține următoarea imagine:

